
ATTRIBUTE-DRIVEN DESIGN DOCUMENT

for

LKFORUM WEBSITE

Version 1.4

Prepared by : Quach Gia Kiet

Submitted to : Nguyen Trinh Dong
Lecturer

May 12, 2026

Contents

1	Design Constraints	3
2	Quality Attribute Requirements	4
2.1	Performance	4
2.1.1	High-Throughput Content Retrieval	4
2.1.2	Asynchronous Processing	4
2.2	Security	5
2.2.1	Authentication and Authorization	5
2.2.2	Data Privacy (End-to-End Encryption)	5
2.3	Availability	6
2.3.1	Container Fault Recovery	6
2.3.2	Resilience Against Load Spikes	6
2.4	Maintainability	7
2.4.1	Modular Architecture (Separation of Concerns)	7
2.4.2	Technology Agnosticism (Interfaces)	7
2.5	Supportability	8
2.5.1	Audit and User Activity Logging	8
2.5.2	Error Diagnostics	8
2.6	Manageability	9
2.6.1	System Health Monitoring	9
2.6.2	Configuration Management	9
3	Architectural Representation	10
3.1	Logical View	10
3.2	Implementation View	12
3.3	Deployment View	13
3.4	Data View	15

1 Design Constraints

- **Performance:** The system must deliver real-time experiences with low latency. It requires the use of the **Go/Gin** framework for high-concurrency request handling, **Redis caching** to offload database reads, and an **Event-Driven** architecture using in-memory **Go Channels** for non-blocking asynchronous tasks.
- **Security:** The system must protect user data and ensure strict access control. It mandates the use of **JSON Web Tokens (JWT)** for authentication, **Role-Based Access Control (RBAC)** for authorization, and **End-to-End Encryption (E2EE)** to guarantee data privacy in private messaging.
- **Availability:** The platform must maintain high uptime and recover quickly from faults. It requires **Docker containerization** to isolate services and leverage automated restart policies upon failure, relying on a stateless backend design.
- **Maintainability:** The architecture must be easy to modify and extend. It enforces a strict **Layered Architecture** (Controllers, Services, Repositories) and the use of abstract **Interfaces** to ensure high cohesion, loose coupling, and testability.
- **Supportability:** The system must facilitate easy troubleshooting and security auditing. It requires a comprehensive **Logging** mechanism to structurally record all critical user activities and system operations.
- **Manageability:** The system must support proactive health tracking and administration. It mandates a centralized **Monitoring** stack utilizing **Prometheus** to scrape metrics and **Grafana** to visualize real-time performance and trigger alerts.
- **Testability:** The system must support comprehensive automated testing at all levels. It requires the use of **Dependency Injection** via Go Interfaces to easily mock dependencies (e.g., database repositories) during unit testing.
- **Reusability:** The system must promote code reuse to minimize duplication. It mandates the creation of **Shared Utility Packages** (e.g., for error handling, JWT parsing, and pagination) in the backend and a reusable UI Component Library in the Svelte frontend.
- **Scalability:** The platform must be capable of handling future growth in user traffic. It requires a **Stateless Architecture** allowing the backend to scale horizontally across multiple instances behind a load balancer.

2 Quality Attribute Requirements

2.1 Performance

2.1.1 High-Throughput Content Retrieval

Element	Statement
Stimulus	Thousands of users simultaneously request the global public feed.
Stimulus source	Guest Users, Members.
Environment	Peak usage times causing high read traffic.
Artifact	Post Service, Redis Cache Layer, MongoDB.
Response	The system serves feeds directly from Redis RAM using a Read-Through caching strategy, bypassing the database completely for cached endpoints.
Response measure	Database CPU utilization remains < 60% during peak loads. $\geq$ 95% of feed read requests process within 500ms.

2.1.2 Asynchronous Processing

Element	Statement
Stimulus	A user publishes a new post that requires notifying hundreds of followers and updating analytics.
Stimulus source	Members.
Environment	Normal operation during content creation.
Artifact	Post Service, Notification Service, In-Memory Event Bus (Go Channels).
Response	The Post Service saves the content and immediately returns success to the user. It then asynchronously publishes an event to the Event Bus via Go Channels. Background goroutines consume the event and process notifications.
Response measure	The HTTP response time for post creation remains < 1 second, unaffected by the number of followers receiving notifications.

2.2 Security

2.2.1 Authentication and Authorization

Element	Statement
Stimulus	Users attempt to access restricted API endpoints, such as deleting a post or changing community settings.
Stimulus source	Malicious Actors, Members, System Admins.
Environment	Normal operation over public internet.
Artifact	JWT Validation Middleware, RBAC Module.
Response	The system validates JWT signatures and strictly enforces RBAC rules. It checks if the authenticated user holds the required role (e.g., Moderator or Admin) before allowing the action.
Response measure	100% of unauthorized API requests are rejected with HTTP 403 Forbidden.

2.2.2 Data Privacy (End-to-End Encryption)

Element	Statement
Stimulus	Two users initiate a private one-to-one conversation via direct messages.
Stimulus source	Members.
Environment	Direct messaging via WebSocket connections.
Artifact	Client-side Cryptography Module (Svelte), Messaging Service, MongoDB.
Response	Messages are encrypted on the sender's client using the recipient's public key (End-to-End Encryption). The backend stores and routes the cipher-text without ever possessing the decryption keys.
Response measure	0% of private message plain-text is visible or readable by server administrators or via database dumps.

2.3 Availability

2.3.1 Container Fault Recovery

Element	Statement
Stimulus	The primary backend application container crashes due to an unhandled exception or memory leak.
Stimulus source	Internal software fault.
Environment	Production environment under normal load.
Artifact	Docker Engine, Stateless Go Backend Containers.
Response	Docker's restart policy detects the container failure and automatically restarts the service. The stateless nature of the backend ensures no user session data is lost (stored in Redis).
Response measure	The system recovers and accepts new requests within 30 seconds. Overall system uptime remains $\geq 99.9\%$.

2.3.2 Resilience Against Load Spikes

Element	Statement
Stimulus	A sudden influx of users log in simultaneously during a viral event, causing a spike in database connections.
Stimulus source	Viral content causing traffic surge.
Environment	Production environment under sudden peak load.
Artifact	Gin Framework, MongoDB Connection Pool.
Response	The Go/Gin server handles high concurrency using lightweight goroutines. The MongoDB connection pool limits active connections, queueing excess requests instead of crashing the database.
Response measure	The system avoids total failure. $\leq 0.5\%$ error rate due to connection timeouts.

2.4 Maintainability

2.4.1 Modular Architecture (Separation of Concerns)

Element	Statement
Stimulus	A developer needs to update the business logic for calculating user karma points.
Stimulus source	Development Team.
Environment	Development and Maintenance phases.
Artifact	Layered Architecture (Controllers, Services, Repositories).
Response	The developer modifies the karma calculation logic exclusively within the <code>UserService</code> . No changes are required in the HTTP <code>UserController</code> or the MongoDB <code>UserRepository</code> .
Response measure	Code changes are strictly localized. Regression rate after the update is $\leq 1\%$.

2.4.2 Technology Agnosticism (Interfaces)

Element	Statement
Stimulus	The team decides to switch the cloud storage provider for user avatars from Cloudinary to AWS S3.
Stimulus source	Infrastructure Team.
Environment	Development and Maintenance phases.
Artifact	Go Interfaces (e.g., <code>ImageStorageProvider</code>).
Response	Developers create a new struct <code>S3StorageProvider</code> that implements the existing <code>ImageStorageProvider</code> interface. Dependency injection is used to wire the new provider at startup.
Response measure	0 changes are required in the core business logic (Services) to swap the storage provider.

2.5 Supportability

2.5.1 Audit and User Activity Logging

Element	Statement
Stimulus	A system administrator needs to trace the action history of a suspected malicious user.
Stimulus source	System Administrators, Security Auditors.
Environment	Post-incident investigation.
Artifact	Structured Logging Middleware, Centralized Log Storage.
Response	The administrator queries the logs. The system provides structured, searchable entries containing contextual metadata (User ID, IP, Action, Timestamp) for every critical state change.
Response measure	100% of critical user activities (login, post deletion, role assignment) are retrievable.

2.5.2 Error Diagnostics

Element	Statement
Stimulus	A user reports an unexpected HTTP 500 Internal Server Error when attempting to vote on a poll.
Stimulus source	Members.
Environment	Production environment during fault condition.
Artifact	Global Error Handler, slog (Structured Logging).
Response	The backend catches the panic or error, returns a generic 500 error to the client to hide internal details, but logs a complete stack trace and request payload to the server logs.
Response measure	The root cause can be identified by developers within 15 minutes by examining the structured error logs.

2.6 Manageability

2.6.1 System Health Monitoring

Element	Statement
Stimulus	The backend experiences an unexpected spike in HTTP 500 errors or increased response latency.
Stimulus source	Unpredictable traffic surge or internal bottleneck.
Environment	Production runtime.
Artifact	Prometheus Scraper, Grafana Dashboards, Go <code>/metrics</code> endpoint.
Response	Prometheus scrapes the elevated error rates from the backend. Grafana evaluates the alert rules and immediately triggers a notification to the operations team via email/Slack.
Response measure	The operations team is alerted within 1 minute of the threshold violation. The dashboard visualizes the exact endpoint causing the issue.

2.6.2 Configuration Management

Element	Statement
Stimulus	The operations team needs to rotate the JWT Secret Key or change the database URI for a staging environment.
Stimulus source	Operations Team.
Environment	Deployment phase.
Artifact	Environment Variables (<code>.env</code>), Viper Config Library.
Response	The system reads all critical configurations from environment variables at startup. The team updates the variables in the <code>docker-compose.yml</code> and restarts the container without modifying or recompiling the code.
Response measure	Configuration changes take effect immediately upon container restart. 0 code recompilations are required for environment-specific settings.

2.7 Testability

2.7.1 Unit Testing with Mocks

Element	Statement
Stimulus	A developer writes unit tests for the core business logic in the <code>PostService</code> .
Stimulus source	Development Team.
Environment	Local development and CI pipeline.
Artifact	Go Interfaces, Mockgen Tool, <code>PostService</code> .
Response	The developer generates mock implementations of the <code>PostRepository</code> interface and injects them into the <code>PostService</code> , allowing testing of business logic without a live database.
Response measure	Unit tests execute in < 1 second. Test coverage for the Service layer reaches $\geq 80\%$.

2.7.2 Integration Testing

Element	Statement
Stimulus	A developer submits a Pull Request, triggering the automated CI/CD pipeline.
Stimulus source	CI/CD System (e.g., GitHub Actions).
Environment	Automated build environment.
Artifact	Docker, Go Testing Framework.
Response	The CI pipeline spins up ephemeral Docker containers for MongoDB and Redis, runs the full integration test suite against the API endpoints, and destroys the containers upon completion.
Response measure	Integration tests complete in < 5 minutes. 100% of critical API paths are automatically verified before merging.

2.8 Reusability

2.8.1 Shared Backend Utilities

Element	Statement
Stimulus	A developer is assigned to build a new feature that requires returning paginated data to the client.
Stimulus source	Development Team.
Environment	Development phase.
Artifact	Shared Utility Packages (<code>pkg/pagination</code>).
Response	The developer imports the existing <code>pagination</code> package instead of writing custom logic, ensuring consistent API responses across all endpoints.
Response measure	0 lines of duplicated pagination logic are introduced into the codebase. Development time is reduced by 20%.

2.8.2 Frontend Component Library

Element	Statement
Stimulus	A frontend developer needs to create a new page containing user avatar icons and standard buttons.
Stimulus source	Frontend Development Team.
Environment	Development phase.
Artifact	Svelte UI Components (<code>Avatar.svelte</code> , <code>Button.svelte</code>).
Response	The developer reuses the standardized Svelte components, maintaining consistent styling and reducing redundant CSS/HTML.
Response measure	UI consistency is maintained at 100%. Redundant code is reduced, leading to smaller bundle sizes.

2.9 Scalability

2.9.1 Horizontal Backend Scaling

Element	Statement
Stimulus	User traffic increases by 500% over the course of a week due to a successful marketing campaign.
Stimulus source	Sudden increase in active users.
Environment	Production environment under heavy load.
Artifact	Stateless Go Backend, Reverse Proxy/Load Balancer.
Response	The operations team scales the backend by spinning up multiple instances of the Go API container. The Load Balancer distributes incoming requests evenly across all instances.
Response measure	System throughput scales linearly with the number of instances. End-user latency remains < 1 second despite the 500% traffic increase.

2.9.2 Database Read Scaling

Element	Statement
Stimulus	The primary MongoDB instance approaches 90% CPU utilization due to a high volume of read queries.
Stimulus source	High volume of concurrent users.
Environment	Production database.
Artifact	MongoDB Replica Set, Redis Cache.
Response	The system offloads repetitive read queries to the Redis cache. If needed, the operations team can configure MongoDB Read Replicas to distribute the read operations across multiple secondary database nodes.
Response measure	Primary database CPU utilization drops below 50%. Read query throughput increases significantly without affecting write operations.

3 Architectural Representation

To describe the architecture of the LKForum System, the following views are presented:

3.1 Logical View

This view presents the system's modular decomposition into functional components and subsystems. It aligns with the **System Context** of the C4 model.

System Context Diagram - LKForum

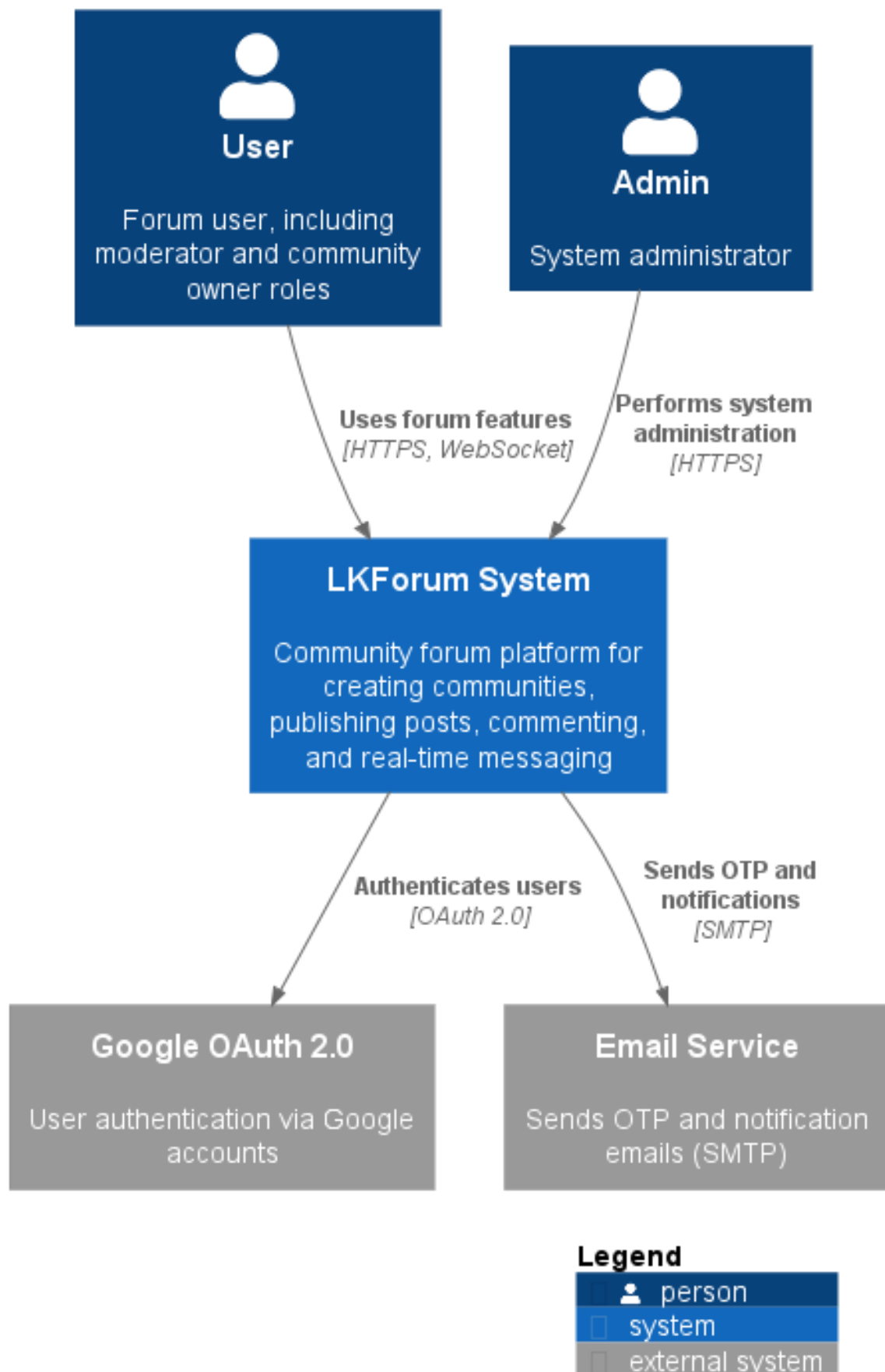


Figure 3.1: System Context Diagram

- **Layered Architecture & Interfaces:** The backend strictly separates concerns into Routes, Controllers, Services, and Repositories. Interfaces define contracts between layers (e.g., `PostService` interface), allowing for easy mock generation and unit testing.
- **IAM (Identity & Access Management) Module:** Handles user registration, JWT-based stateless authentication, Google OAuth SSO integration, and RBAC.
- **Community & Content Modules:** Manages the lifecycle of sub-forums, posts, and nested comments.
- **Event-Driven Hub:** An internal message bus utilizing **Go Channels and Goroutines** that decouples core business logic from auxiliary tasks (like firing push notifications or updating search indexes), ensuring high performance without relying on external message brokers.
- **Communication Module:** Facilitates real-time features including E2EE private messaging via WebSockets.

3.2 Implementation View

This view outlines the organization of code and deployment artifacts, aligning with the **Component Diagram** of the C4 model.

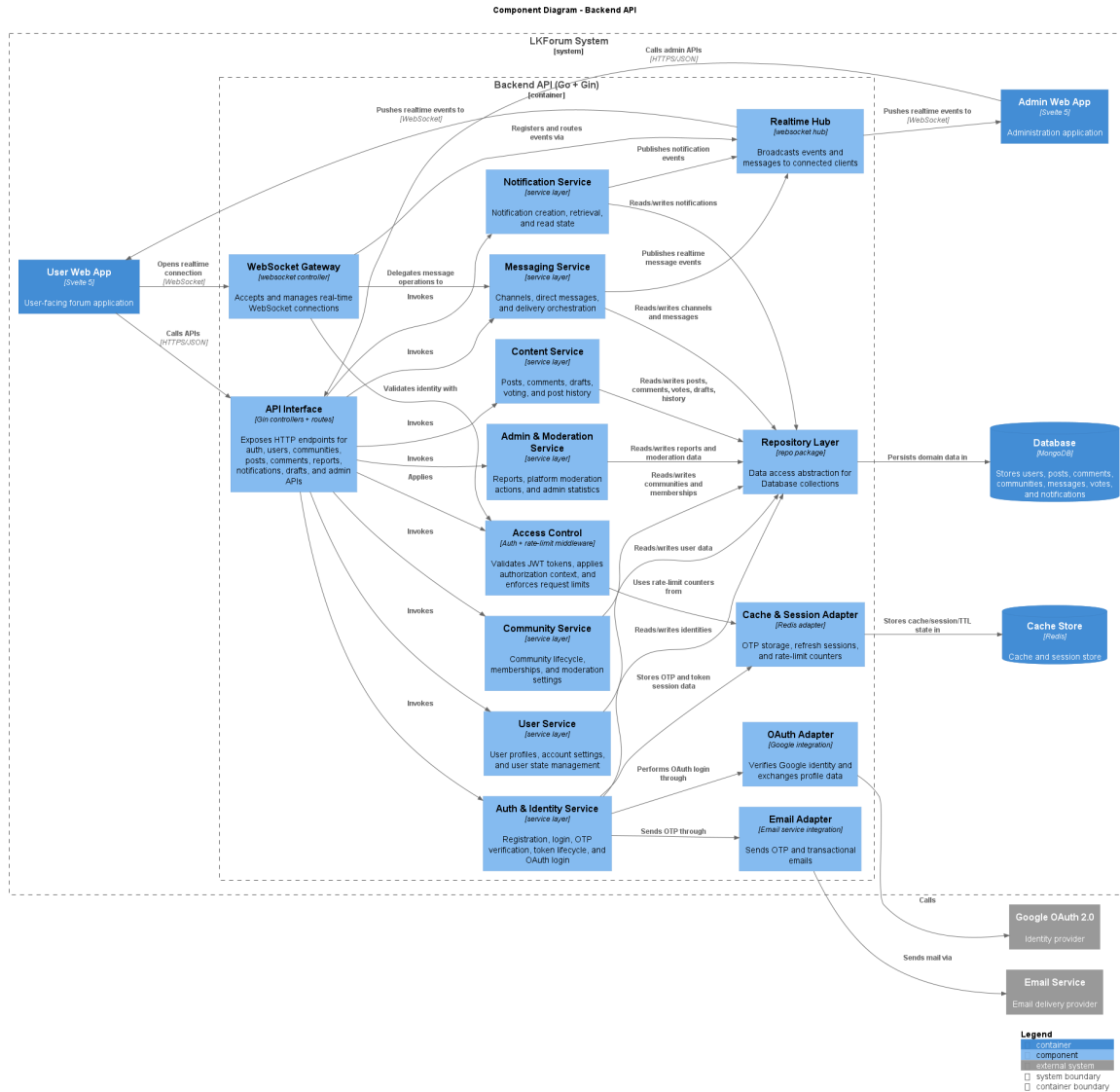


Figure 3.2: Backend Component Diagram

- **Backend Organization:** The backend is a monolithic Go application exposing REST APIs and WebSocket endpoints.
- **Frontend Organization:** Separated into two distinct SPAs built with Svelte and Vite:
 - **user-web:** The main interface for Guests, Members, and Community Moderators.
 - **admin-web:** A dedicated dashboard for System Administrators.
- **Observability Implementation:** The backend integrates Prometheus client libraries to expose metrics and structured logging (e.g., using `slog` or `logrus`) for user activity tracking.

3.3 Deployment View

Describes the physical deployment and runtime environment of system components, aligning with the **Container Diagram** of the C4 model.

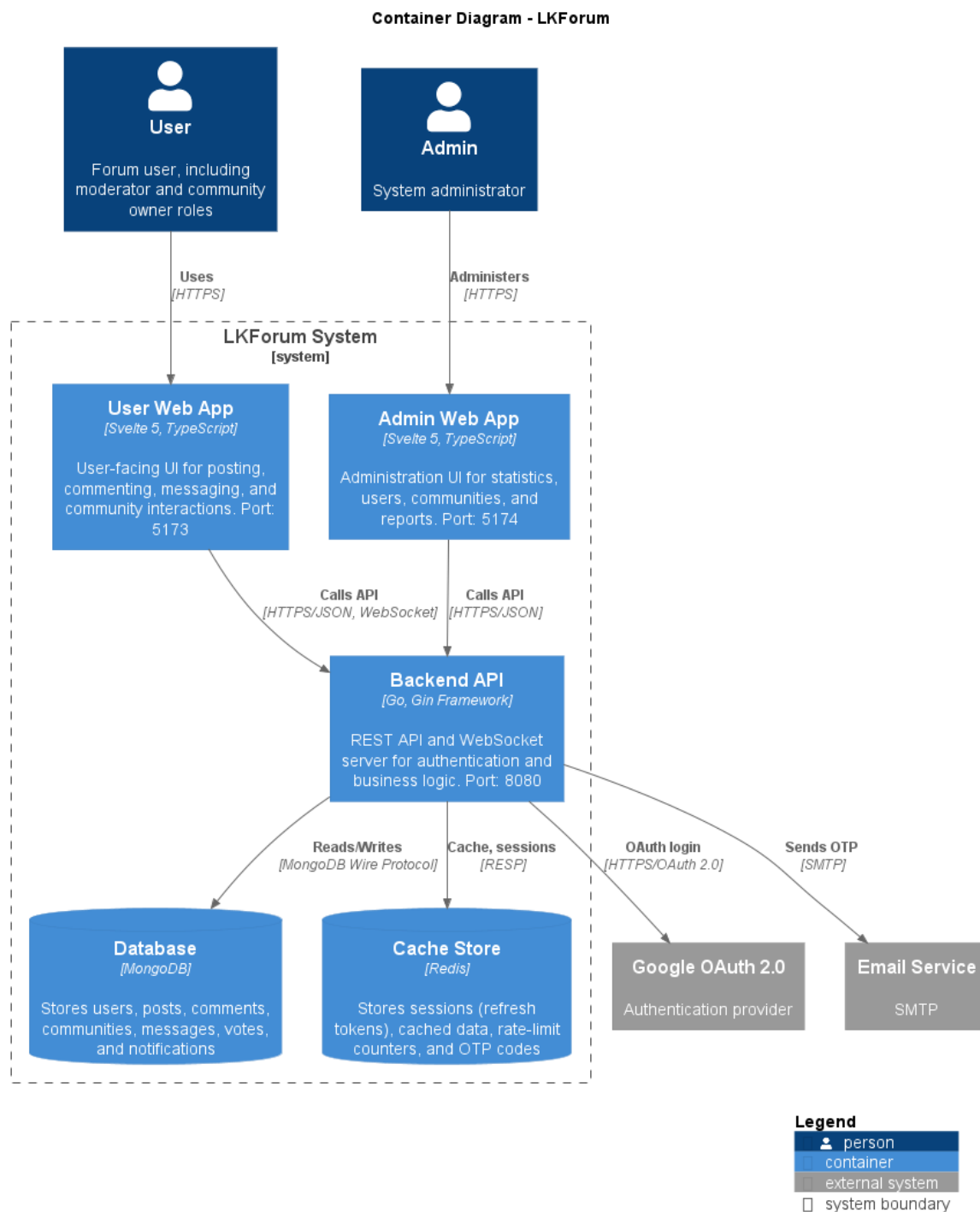


Figure 3.3: Container Diagram

- **Containerization:** The entire ecosystem is containerized using Docker, avoiding the overhead of Kubernetes while maintaining isolation.
- **Orchestration:** A `docker-compose.yml` file manages the multi-container application network.
- **Node Layout:**
 - **Application Nodes:** Go Backend Container, User Web Frontend Container, Admin Web

Frontend Container.

- **Data Nodes:** MongoDB container (persistent data), Redis container (caching).
- **Monitoring Nodes:** Prometheus container (metrics scraping) and Grafana container (dashboards).

3.4 Data View

Focuses on data models, storage strategy, and external data integrations.

- **Primary Storage (MongoDB):** Chosen for its flexible schema design, suitable for dynamic content types (polls, media arrays).
- **Caching (Redis):** Acts as an in-memory cache to absorb high read loads for public feeds, and stores fast-expiring data like session tokens and OTPs.
- **Secure Messaging Storage:** Private messages stored in MongoDB are encrypted blobs; the server routes them but relies on client devices for actual decryption (E2EE).